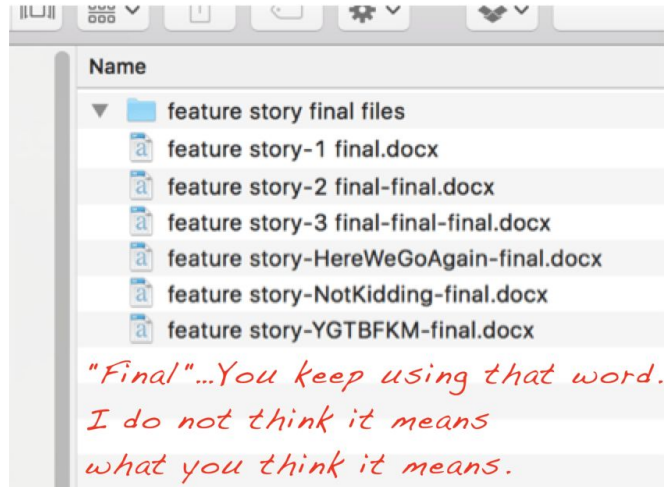


Git

Einführung in die Versionsverwaltung git

Einführung

Wer kennt es nicht?



Irgendwie sind wir nie wirklich fertig.

Versionskontrolle

- Versionskontrollsysteme speichern intern verschiedene Versionen einer Datei
- Versionen sind nach Änderungen zeitlich sortiert
- Man überschreibt immer die selbe Datei, und überlässt dem VKS das verwalten der verschiedenen Revisionen
- Ältere Revisionen lassen sich über das VKS aufrufen
- Auch verteiltes Arbeiten mit mehreren Personen ist möglich

Meistgenutztes VKS: git.

Über diese Vorlesung/Übung

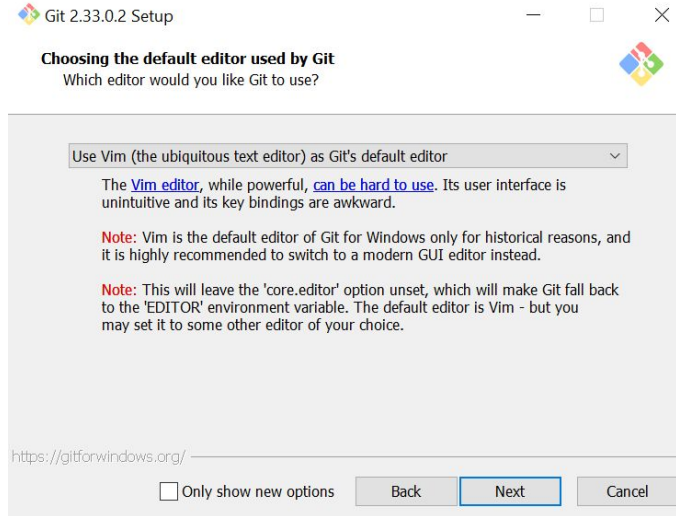
Während dieser Vorlesung werden einzelne Schritte aufgezeigt. Jeder Schritt soll sofort durch euch durchgeführt werden: Kombinierte Vorlesung und Übung.

Material: Freies Ebook Pro Git: <https://git-scm.com/book/en/v2>

Ich empfehle die englische Version, eine Deutsche ist aber auch vorhanden:
<https://git-scm.com/book/de/v2>

Git Installieren

Bitte installiert nun git: <https://git-scm.com/downloads> (10 Minuten Zeit)



In der Vorlesung benutze ich Vim

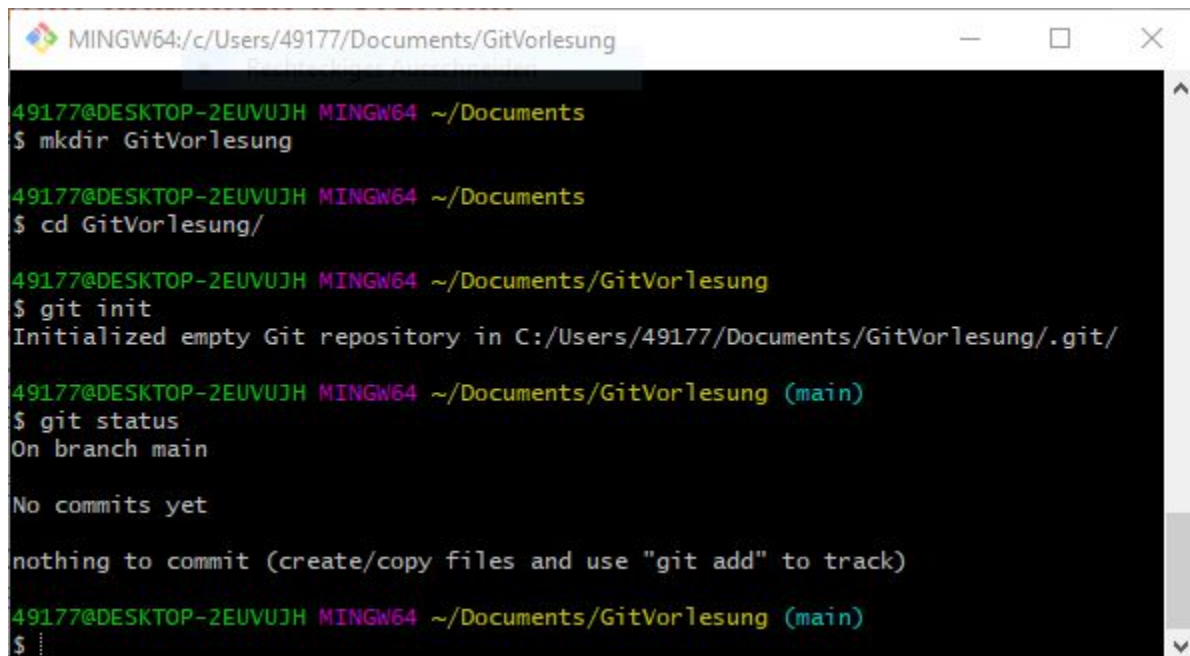
Erstellen eines git Repositories

- Git verwaltet Änderungen von Dateien in einem Ordner.
- Um git mitzuteilen, dass es Dateien in einem Ordner verwalten soll, muss ein Repository in diesem Ordner erzeugt werden.
- Das Repository befindet sich nach der Erzeugung im **.git** Unterordner des ausgewählten Ordners und wird über **git** Befehle angesprochen.
- Zum erstellen des Repositories benutzen wir den **git init** Befehl

Erstelle nun ein Repository in einem neuen Order

- Öffne Git Bash oder Terminal
- Erstelle einen neuen Order
- Wechsle in diesen Ordner
- Führe **git init** aus
- Überprüfe mit **git status**
- Zeit: 5 Minuten

Resultat



```
MINGW64:/c/Users/49177/Documents/GitVorlesung
49177@DESKTOP-2EUVUJH MINGW64 ~/Documents
$ mkdir GitVorlesung

49177@DESKTOP-2EUVUJH MINGW64 ~/Documents
$ cd GitVorlesung/

49177@DESKTOP-2EUVUJH MINGW64 ~/Documents/GitVorlesung
$ git init
Initialized empty Git repository in C:/Users/49177/Documents/GitVorlesung/.git/

49177@DESKTOP-2EUVUJH MINGW64 ~/Documents/GitVorlesung (main)
$ git status
On branch main

No commits yet

nothing to commit (create/copy files and use "git add" to track)

49177@DESKTOP-2EUVUJH MINGW64 ~/Documents/GitVorlesung (main)
$
```


Erzeugen einer Revision (Commit)

- Um eine Revision zu erzeugen, erstellen wir zunächst eine Textdatei und füllen sie mit Inhalt
- Anschließend fügen wir die neue Datei mit dem `git add <filename>` Befehl zur Versionskontrolle hinzu
- Nun speichern wir die Revision unserer Datei mit dem `git commit` Befehl
- Einer Revision wird eine Beschreibung hinzugefügt. `git commit` öffnet für uns einen Editor.
- Für Vim-Nutzer: Drücke `i` zum schreiben, und `<Esc>` wenn du fertig bist. Speichere und schließe indem du `:x` eingibst und dann `<Return>` drückst.

Erzeugen einer Revision (Commit)

- Erzeuge eine Datei `content.txt` und füge einen Text mit mehreren Zeilen und Leerzeilen in die Datei ein
- Füge die Datei zu git hinzu
- Erstelle einen Commit mit der Message "Initial commit"
- Zeit: 10 Minuten

Resultat

```
MINGW64:/c/Users/49177/Documents/GitVorlesung
49177@DESKTOP-2EUVUJH MINGW64 ~/Documents/GitVorlesung (main)
$ vim content.txt

49177@DESKTOP-2EUVUJH MINGW64 ~/Documents/GitVorlesung (main)
$ git add content.txt

49177@DESKTOP-2EUVUJH MINGW64 ~/Documents/GitVorlesung (main)
$ git status
On branch main

No commits yet

Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
    new file:   content.txt

49177@DESKTOP-2EUVUJH MINGW64 ~/Documents/GitVorlesung (main)
$ git commit
[main (root-commit) ae513e1] Initial commit
 1 file changed, 4 insertions(+)
 create mode 100644 content.txt

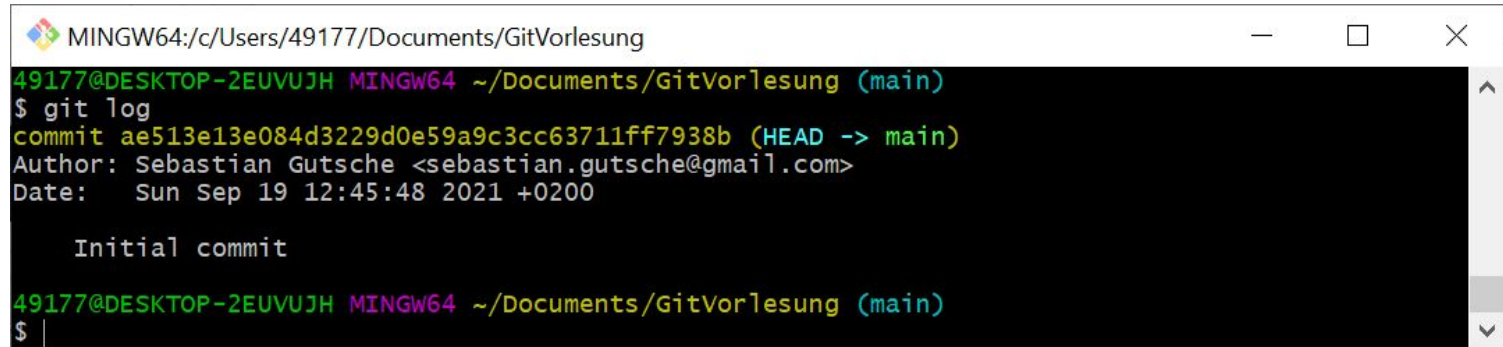
49177@DESKTOP-2EUVUJH MINGW64 ~/Documents/GitVorlesung (main)
$ git status
On branch main
nothing to commit, working tree clean
```

Nützliches

- `git commit -a` fügt alle Dateien zum Commit hinzu, `git add` entfällt
- `git commit --ammend` fügt Änderungen zum aktuellen Commit hinzu
- `git add -ip <filename>` erlaubt das partielle Committen von Änderungen in einer Datei

Log und Diff

Das git log ermöglicht es uns, den Verlauf unserer Änderungen zu sehen

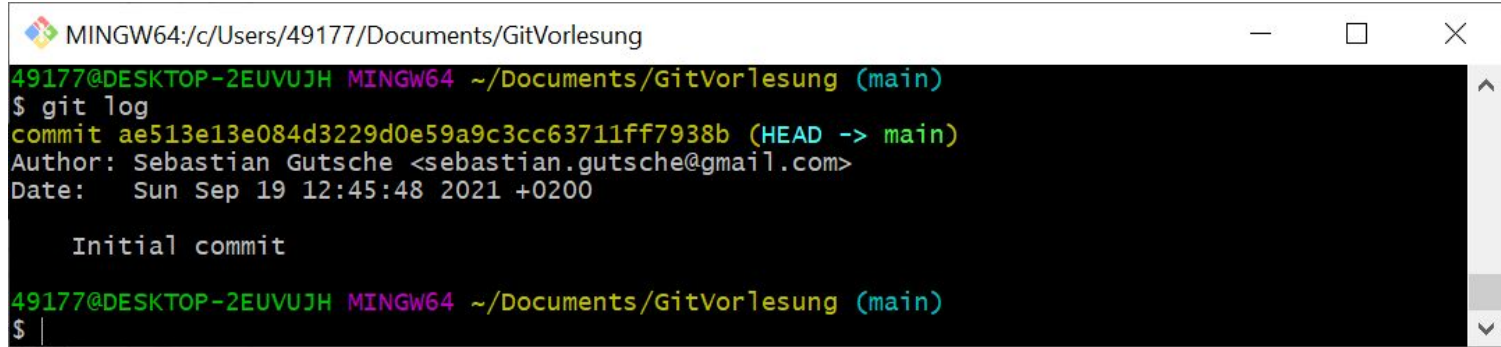


```
MINGW64:/c/Users/49177/Documents/GitVorlesung
49177@DESKTOP-2EUVUJH MINGW64 ~/Documents/GitVorlesung (main)
$ git log
commit ae513e13e084d3229d0e59a9c3cc63711ff7938b (HEAD -> main)
Author: Sebastian Gutsche <sebastian.gutsche@gmail.com>
Date:   Sun Sep 19 12:45:48 2021 +0200

    Initial commit

49177@DESKTOP-2EUVUJH MINGW64 ~/Documents/GitVorlesung (main)
$ |
```

Struktur des Commits



```
MINGW64:/c/Users/49177/Documents/GitVorlesung
49177@DESKTOP-2EUVUJH MINGW64 ~/Documents/GitVorlesung (main)
$ git log
commit ae513e13e084d3229d0e59a9c3cc63711ff7938b (HEAD -> main)
Author: Sebastian Gutsche <sebastian.gutsche@gmail.com>
Date:   Sun Sep 19 12:45:48 2021 +0200

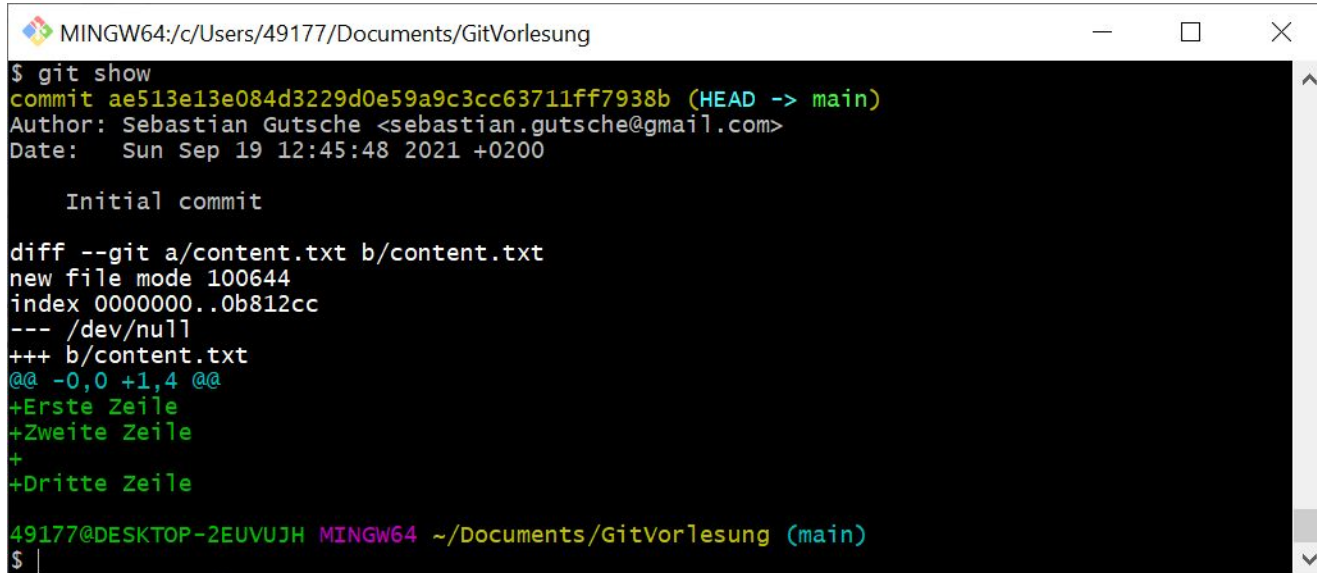
    Initial commit

49177@DESKTOP-2EUVUJH MINGW64 ~/Documents/GitVorlesung (main)
$ |
```

- Commit ID: Referenziert eindeutig auf den Commit
- Author & Date: Metainformation
- Message: Sollte möglichst gute Beschreibung der Änderungen enthalten

Log und Diff

Ein Diff in git ist die Menge an Änderungen in einem Commit. Mit `git show` sehen wir die Änderungen im letzten Commit:



```
MINGW64:/c/Users/49177/Documents/GitVorlesung
$ git show
commit ae513e13e084d3229d0e59a9c3cc63711ff7938b (HEAD -> main)
Author: Sebastian Gutsche <sebastian.gutsche@gmail.com>
Date:   Sun Sep 19 12:45:48 2021 +0200

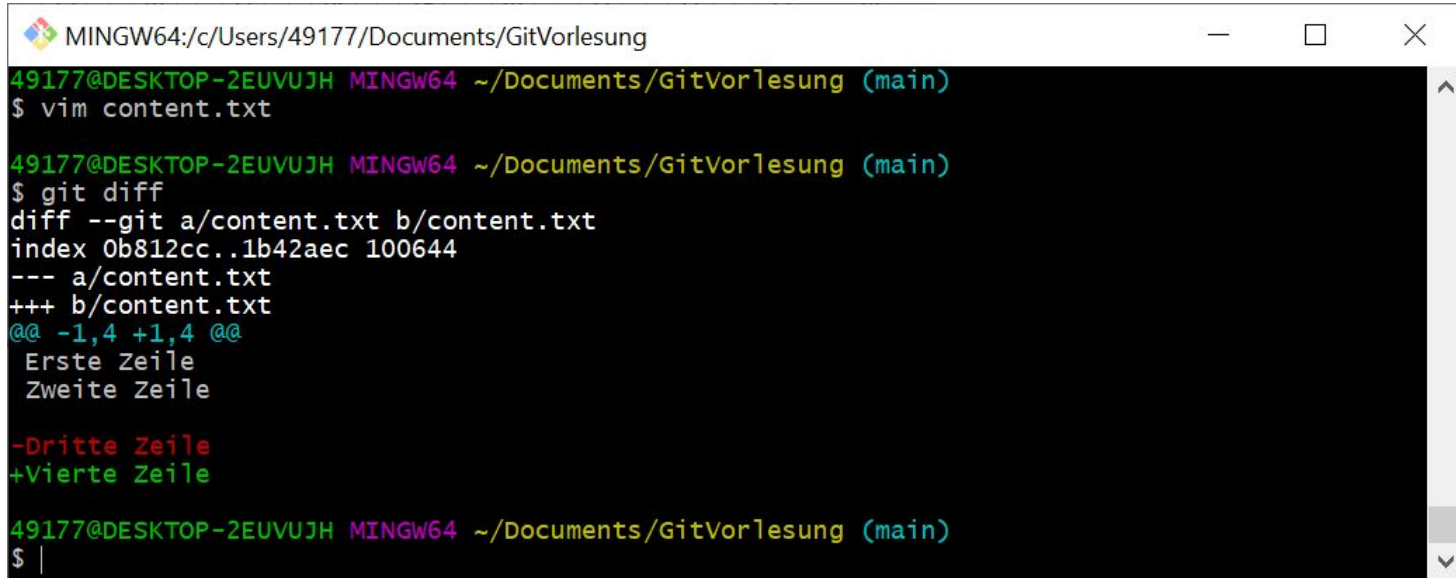
    Initial commit

diff --git a/content.txt b/content.txt
new file mode 100644
index 00000000..0b812cc
--- /dev/null
+++ b/content.txt
@@ -0,0 +1,4 @@
+Erste Zeile
+Zweite Zeile
+
+Dritte Zeile

49177@DESKTOP-2EUVUJH MINGW64 ~/Documents/GitVorlesung (main)
$
```

Log und Diff

Mit `git diff` sehen wir die Änderungen seit dem letzten Commit



```
MINGW64:/c/Users/49177/Documents/GitVorlesung
49177@DESKTOP-2EUVUJH MINGW64 ~/Documents/GitVorlesung (main)
$ vim content.txt

49177@DESKTOP-2EUVUJH MINGW64 ~/Documents/GitVorlesung (main)
$ git diff
diff --git a/content.txt b/content.txt
index 0b812cc..1b42aec 100644
--- a/content.txt
+++ b/content.txt
@@ -1,4 +1,4 @@
  Erste Zeile
  Zweite Zeile

-Dritte Zeile
+Vierte Zeile

49177@DESKTOP-2EUVUJH MINGW64 ~/Documents/GitVorlesung (main)
$ |
```


Nützliches

- `git reset --hard` setzt das Repository auf den letzten Commit zurück

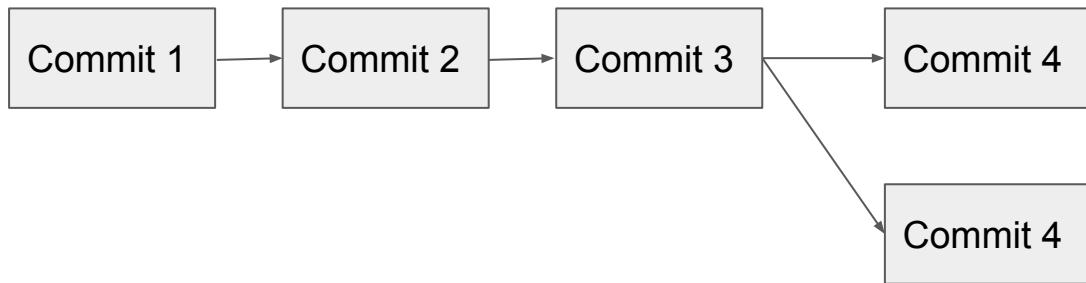
Pause

bis 9:30

Branches

Manchmal ist es notwendig, an verschiedenen Stellen im Projekt gleichzeitig an unterschiedlichen Dingen zu Arbeiten.

Um schnell zwischen diesen Dingen hin und her zu schalten, können branches benutzt werden.



Branches

- Branches sind verschiedene Änderungen der gleichen Revision, die parallel laufen.
- Um einen neuen Branch zu erstellen, benutze **git branch <branch_name>**
- Um auf den neuen Branch zu wechseln, benutze **git checkout <branch_name>**
- Nun kann auf dem neuen Branch committed werden
- Der Name des default branches ist “main”

Branches

- Erzeuge einen neuen Branch “feature”
- Wechsle auf diesen Branch
- Füge Text zu context.txt hinzu
- Committe deine Änderungen
- Wechsle zurück zu “main”
- Überprüfe die Datei
- Zeit: 10 Minuten

Resultat

```
MINGW64:/c/Users/49177/Documents/GitVorlesung

49177@DESKTOP-2EUVUJH MINGW64 ~/Documents/GitVorlesung (main)
$ git branch feature

49177@DESKTOP-2EUVUJH MINGW64 ~/Documents/GitVorlesung (main)
$ git checkout feature
Switched to branch 'feature'

49177@DESKTOP-2EUVUJH MINGW64 ~/Documents/GitVorlesung (feature)
$ vim content.txt

49177@DESKTOP-2EUVUJH MINGW64 ~/Documents/GitVorlesung (feature)
$ git add content.txt

49177@DESKTOP-2EUVUJH MINGW64 ~/Documents/GitVorlesung (feature)
$ git commit
[feature 4bceff2] Second commit
1 file changed, 2 insertions(+)

49177@DESKTOP-2EUVUJH MINGW64 ~/Documents/GitVorlesung (feature)
$ git shortlog
Sebastian Gutsche (2):
    Initial commit
    Second commit

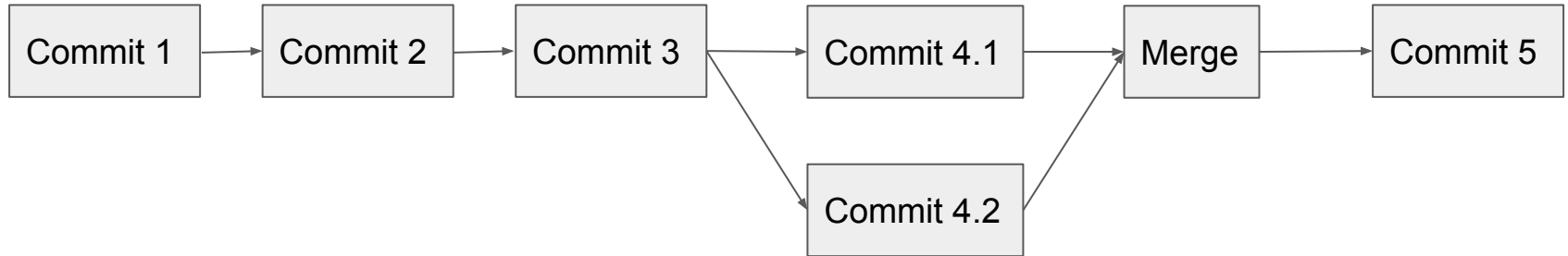
49177@DESKTOP-2EUVUJH MINGW64 ~/Documents/GitVorlesung (feature)
$ git checkout main
Switched to branch 'main'

49177@DESKTOP-2EUVUJH MINGW64 ~/Documents/GitVorlesung (main)
$ git shortlog
Sebastian Gutsche (1):
    Initial commit

49177@DESKTOP-2EUVUJH MINGW64 ~/Documents/GitVorlesung (main)
$
```

Merging

Nachdem Änderungen auf anderen Branches gemacht werden, müssen diese gemerged werden



Merging

- Ein Merge Commit ist ein Commit der anzeigt, an welcher Stelle Änderungen zusammengefügt wurden
- Er enthält selbst keine Änderungen
- Commits von beiden Branches bleiben erhalten
- Man merged Änderungen von einer anderen Branch auf die aktuelle. Ist man auf der Branch main, so kann man die Änderungen von feature mit **git merge feature** mergen

Merging

Generell gibt es zwei Möglichkeiten des Mergens:

- Fast-Forward: Ist die Branch von der man merged eine Fortsetzung der aktuellen Branch, so werden nur die neuen Commits übertragen
- Merge-with-commit: Falls die Branches divergieren, so wird ein Merge Commit erzeugt.

Merging

- Erstelle zwei Branches, und mache Änderungen auf beiden Branches
- Merge beide nach main. Was kann man sehen?
- Zeit 15 Minuten

Resultat

```
MINGW64:/c/Users/49177/Documents/GitVorlesung

49177@DESKTOP-2EUVUJH MINGW64 ~/Documents/GitVorlesung (feature)
$ git checkout main
Switched to branch 'main'

49177@DESKTOP-2EUVUJH MINGW64 ~/Documents/GitVorlesung (main)
$ git shortlog
Sebastian Gutsche (1):
    Initial commit

49177@DESKTOP-2EUVUJH MINGW64 ~/Documents/GitVorlesung (main)
$ git shortlog
Sebastian Gutsche (1):
    Initial commit

49177@DESKTOP-2EUVUJH MINGW64 ~/Documents/GitVorlesung (main)
$ git merge feature
Updating ae513e1..4bceff2
Fast-forward
 content.txt | 2 ++
 1 file changed, 2 insertions(+)

49177@DESKTOP-2EUVUJH MINGW64 ~/Documents/GitVorlesung (main)
$ git shortlog
Sebastian Gutsche (2):
    Initial commit
    Second commit

49177@DESKTOP-2EUVUJH MINGW64 ~/Documents/GitVorlesung (main)
$
```

Nützliches

- `git checkout -b <new_branch>` erlaubt gleichzeitiges erzeugen und auschecken einer Branch
- `git branch` zeigt Branches in einem Repository an
- `git branch -d <branch>` löscht eine nicht mehr benötigte Branch

Mergekonflikte

Falls Änderungen an der selben Stelle in zwei Branches gemacht wurden, so kann es sein, dass es zu Merge-Konflikten kommt. Diese müssen manuell gelöst werden.

Mergekonflikte

```
MINGW64:/c:/Users/49177/Documents/GitVorlesung
49177@DESKTOP-2EUVUJH MINGW64 ~/Documents/GitVorlesung (main)
$ git show main
commit 76e49a191437fac2960dc13a1d60f2cc0f94d18e (HEAD -> main)
Author: Sebastian Gutsche <sebastian.gutsche@gmail.com>
Date: Sun Sep 19 14:02:38 2021 +0200

    Auch ein konflikt

diff --git a/content.txt b/content.txt
index 73dbfbf..d93d1a8 100644
--- a/content.txt
+++ b/content.txt
@@ -1,4 +1,4 @@
-Erste Zeile
+Erste Zeile auf main branch
Zweite Zeile

Dritte Zeile

49177@DESKTOP-2EUVUJH MINGW64 ~/Documents/GitVorlesung (main)
$ git show conflict
commit af71c17e8e1019f16f81e927fa88641b03b188f1 (conflict)
Author: Sebastian Gutsche <sebastian.gutsche@gmail.com>
Date: Sun Sep 19 14:02:08 2021 +0200

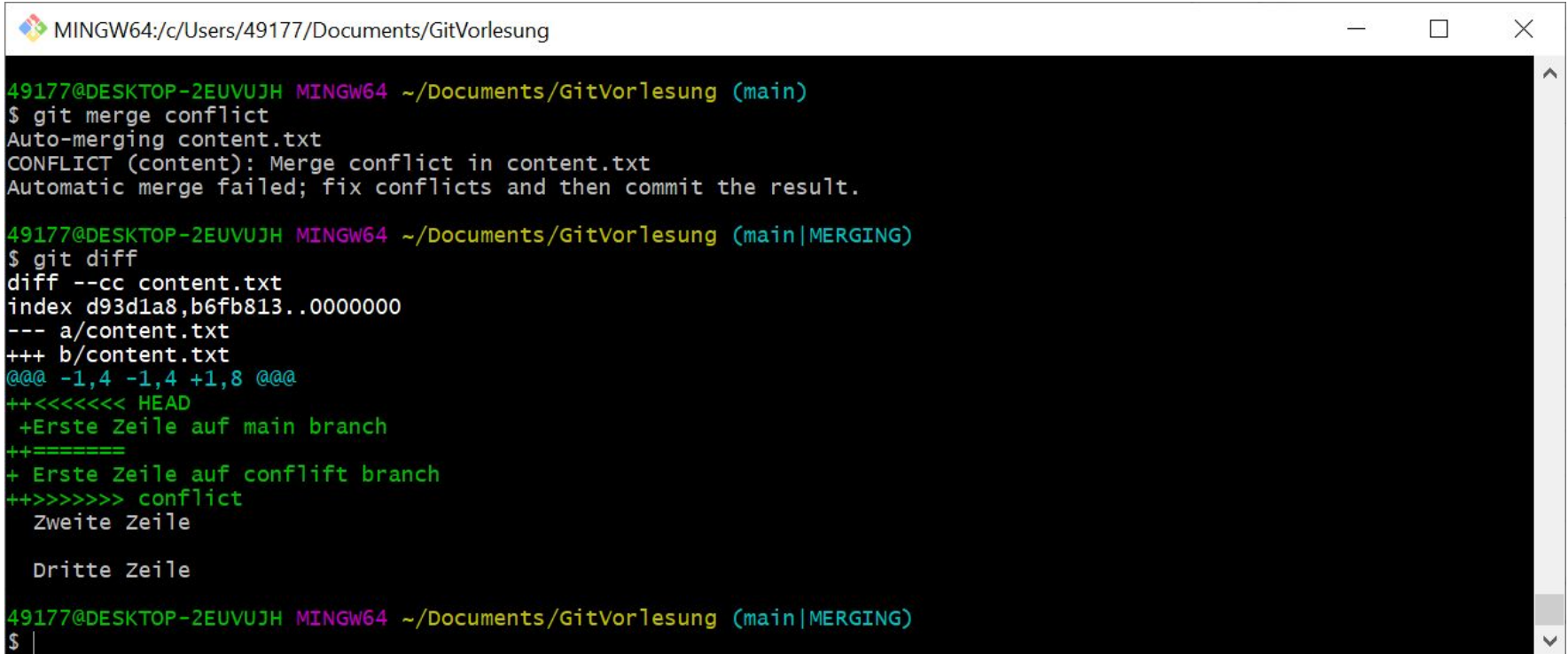
    Ein Konflikt

diff --git a/content.txt b/content.txt
index 73dbfbf..b6fb813 100644
--- a/content.txt
+++ b/content.txt
@@ -1,4 +1,4 @@
-Erste Zeile
+Erste Zeile auf conflict branch
Zweite Zeile

Dritte Zeile

49177@DESKTOP-2EUVUJH MINGW64 ~/Documents/GitVorlesung (main)
$ |
```

Mergekonflikte



```
MINGW64:/c/Users/49177/Documents/GitVorlesung

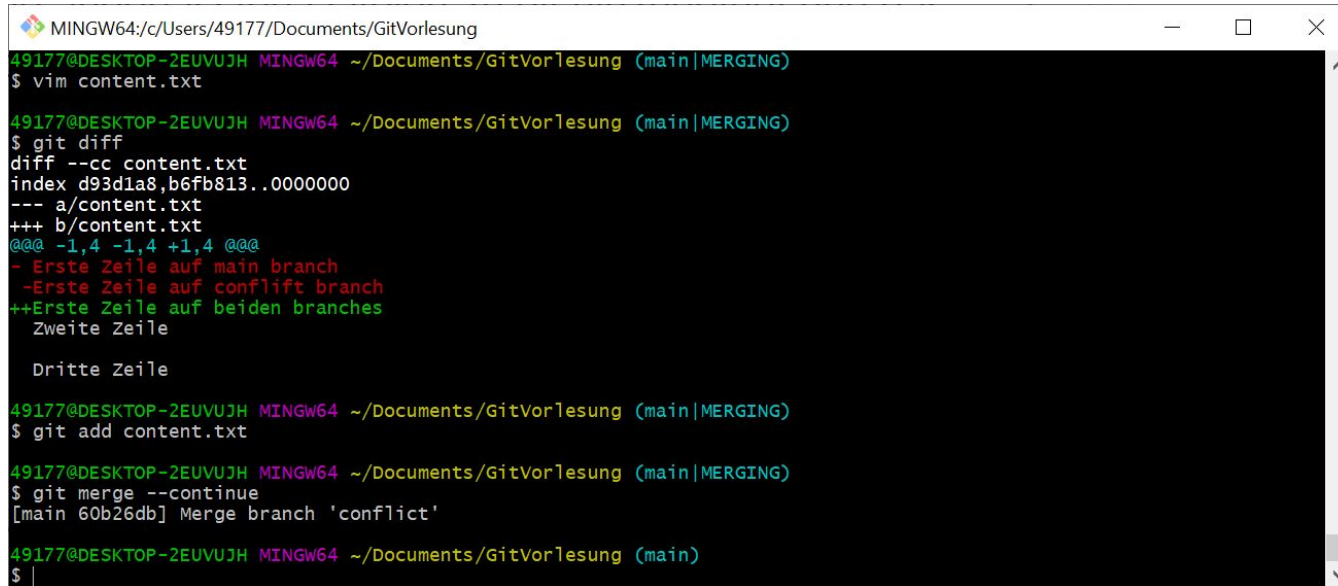
49177@DESKTOP-2EUVUJH MINGW64 ~/Documents/GitVorlesung (main)
$ git merge conflict
Auto-merging content.txt
CONFLICT (content): Merge conflict in content.txt
Automatic merge failed; fix conflicts and then commit the result.

49177@DESKTOP-2EUVUJH MINGW64 ~/Documents/GitVorlesung (main|MERGING)
$ git diff
diff --cc content.txt
index d93d1a8,b6fb813..0000000
--- a/content.txt
+++ b/content.txt
@@@ -1,4 -1,4 +1,8 @@@
++<<<<<< HEAD
+Erste Zeile auf main branch
++=====
+ Erste Zeile auf confligt branch
++>>>>>> conflict
+   Zweite Zeile
+
+   Dritte Zeile

49177@DESKTOP-2EUVUJH MINGW64 ~/Documents/GitVorlesung (main|MERGING)
$ |
```

Mergekonflikte

- Um Mergekonflikte zu lösen, müssen die Dateien manuell angepasst werden
- Anschließend werden die angepassten Dateien zu git hinzugefügt und das mergen wird fortgesetzt



```
MINGW64:/c:/Users/49177/Documents/GitVorlesung
49177@DESKTOP-2EUUVUJH MINGW64 ~/Documents/GitVorlesung (main|MERGING)
$ vim content.txt

49177@DESKTOP-2EUUVUJH MINGW64 ~/Documents/GitVorlesung (main|MERGING)
$ git diff
diff --cc content.txt
index d93d1a8,b6fb813..0000000
--- a/content.txt
+++ b/content.txt
@@@ -1,4 -1,4 +1,4 @@@
- Erste Zeile auf main branch
- Erste Zeile auf conflift branch
++Erste Zeile auf beiden branches
  Zweite Zeile

  Dritte Zeile

49177@DESKTOP-2EUUVUJH MINGW64 ~/Documents/GitVorlesung (main|MERGING)
$ git add content.txt

49177@DESKTOP-2EUUVUJH MINGW64 ~/Documents/GitVorlesung (main|MERGING)
$ git merge --continue
[main 60b26db] Merge branch 'conflict'

49177@DESKTOP-2EUUVUJH MINGW64 ~/Documents/GitVorlesung (main)
$ |
```


Mergekonflikte

- Erzeugt eine Branch mit einem Konflikt
- Merged diese Branch und löst den Mergekonflikt
- Zeit: 10 Minuten

Main & feature branch

- Normalerweise gibt es eine main Branch, von der feature Branches abgehen
- Um Merge Commits zu vermeiden, sollte es auf der main branch nur fast-forward Merges geben
- Hierzu muss vor dem Merge die feature Branch auf den Stand der main Branch gebracht werden
- Dies erfolgt mit dem Befehl `git rebase main` auf der feature Branch
- Hierbei müssen evtl. Mergekonflikte gelöst werden

Weitere git Techniken (zum Selbststudium)

- Bisect
- Interactive rebase
- Remote repositories, shared repositories

Fragen?